

STUDENT MANAGEMENT SYSTEM

- Java + Data Structures Project
- Console-Based Student Record Management Application
- Technology: Java
- Primary Data Structure: ArrayList
- Academic Year: 2026

INTRODUCTION

- Java-based console application for student record management.
- Stores registration number, name, branch, marks, total classes and attended classes.
- Applies Java OOP concepts and collection concepts.
- Student records are represented as Student objects stored in an ArrayList.

PROBLEM STATEMENT

- Manual student records require repetitive searching, updating and calculations.
- Need a simple menu-driven application for common student operations.
- Registration number is used as the main identifier.

EXISTING SYSTEM

- Records may be maintained in notebooks, spreadsheets or separate documents.
- Manual searching is slower for larger numbers of records.
- Repeated calculations can cause inconsistency.
- Changes require additional manual effort.

PROPOSED SYSTEM

- Console-based Java application with menu-driven workflow.
- Stores Student objects in ArrayList.
- Provides login, add, display, search, update, delete, grade, percentage and attendance.
- Student calculations are separated into methods inside Student.

OBJECTIVES

- Store and manage student objects.
- Perform CRUD-style operations.
- Calculate grades and percentages.
- Check attendance eligibility.
- Strengthen Java and ArrayList concepts.

SCOPE

- Runtime student record management through console.
- Authentication and student record operations.
- Grade, percentage and attendance calculation.
- Data is lost when the program terminates.
- Future versions can add database storage.

TECHNOLOGIES & REQUIREMENTS

- Programming Language: Java
- JDK and Java-compatible IDE/compiler.
- ArrayList for storage.
- Scanner for keyboard input.
- No external database or library is required.

SYSTEM ARCHITECTURE

- Student Model: stores student data and calculations.
- StudentManagementSystem Controller: manages collection, input, authentication, menu and operations.
- ArrayList: in-memory storage.
- Scanner: console input.

DATA FLOW

- User enters credentials.
- login() validates credentials.
- Main menu is displayed after successful login.
- Scanner reads the selected operation.
- switch-case routes control to the required method.
- Record operations traverse ArrayList and compare registration numbers.

CLASS DESIGN

- Student: regNo, name, branch, marks, totalClasses, attendedClasses.
- Methods: constructor, getAttendance(), getGrade(), display().
- StudentManagementSystem: ArrayList, Scanner, login(), menu and operation methods.
- Two main classes form the core design.

ARRAYLIST – DSA CONCEPT

- ArrayList is the primary data structure.
- It is a resizable array implementation in Java Collections Framework.
- `students.add()` appends a Student object.
- Enhanced for-loop is used for traversal.
- Sequential lookup makes search/update/delete $O(n)$ worst case.

OOP CONCEPTS

- Class & Object: Student is a class; each record is a Student object.
- Encapsulation: student attributes and methods are grouped in Student.
- Abstraction: `getGrade()` and `getAttendance()` hide calculation details.
- Constructors initialize Student objects.

LOGIN MODULE

- login() asks for username and password using Scanner.
- Current demonstration credentials: student / 1234.
- String.equals() compares credentials.
- Failed login stops the main application.

ADD STUDENT MODULE

- Reads registration number, name, branch, marks, total classes and attended classes.
- Creates a Student object using the constructor.
- Adds the object to ArrayList using add().
- Current version does not prevent duplicate registration numbers.

DISPLAY MODULE

- Checks `students.isEmpty()`.
- If empty, displays No Students Available.
- Otherwise traverses every Student object.
- `display()` shows registration number, name, branch, marks, grade and attendance.

SEARCH MODULE

- Reads registration number.
- Traverses ArrayList sequentially.
- Compares entered regNo with each student's regNo.
- Displays matching record and stops the loop.
- Shows Student Not Found when no match exists.

UPDATE MODULE

- Searches by registration number.
- Updates name, branch and marks.
- Registration number, total classes and attended classes remain unchanged.
- Displays a success message.

DELETE MODULE

- Searches for the requested registration number.
- Removes the matching Student object from ArrayList.
- Returns immediately after deletion.
- Iterator or index-based removal can make modification during traversal more explicit.

GRADE MODULE

- Finds student using registration number.
- 90 or above → A
- 80–89 → B | 70–79 → C | 60–69 → D | below 60 → F
- Grading logic is centralized in `Student.getGrade()`.

PERCENTAGE MODULE

- Current implementation assumes marks are already out of 100.
- Percentage is assigned directly from marks.
- Example: marks 88 → percentage 88%.
- Future formula: $\text{marksObtained} / \text{maximumMarks} \times 100$.

ATTENDANCE MODULE

- Finds student by registration number.
- Attendance = $\text{attendedClasses} / \text{totalClasses} \times 100$.
- 75% or above → Eligible.
- Below 75% → Not Eligible.

MAIN MENU & CONTROL FLOW

- login() is called first.
- do-while loop repeatedly displays the menu.
- switch-case processes choices.
- Choices include Add, Display, Search, Update, Delete, Grade, Percentage, Attendance and Exit.
- Choice 9 terminates the loop.

ALGORITHMS

- Add: read fields → create Student → add to ArrayList.
- Search: read regNo → traverse list → compare regNo → display result.
- Grade: compare marks with thresholds → return A/B/C/D/F.
- Attendance: calculate percentage → compare with 75 → display eligibility.

COMPLEXITY ANALYSIS

- ArrayList add(): amortized $O(1)$, with occasional resizing.
- Display: $O(n)$.
- Search: $O(n)$ worst case.
- Update/Delete: $O(n)$.
- Grade/Percentage/Attendance: $O(n)$ overall; calculation after lookup is $O(1)$.
- Space complexity: $O(n)$.

INPUT VALIDATION

- Current code uses `nextInt()` for numeric input.
- `InputMismatchException` is not caught.
- Marks should be 0–100.
- Total classes should be greater than zero.
- Attended classes should be between zero and total classes.
- Duplicate registration numbers should be rejected.

TESTING

- Test each menu operation separately and in combinations.
- Positive tests: login, insertion, search, update, deletion, grade, percentage and attendance.
- Negative tests: invalid login, missing registration number, empty list, invalid choice and invalid input formats.

SAMPLE TEST CASES

- Correct login → Login Successful.
- Incorrect login → Invalid Username or Password.
- Valid student → Student Added Successfully.
- Existing registration number → Student details displayed.
- Marks 92 → Grade A.
- Attendance 80/100 → 80% and Eligible.
- Attendance 60/100 → 60% and Not Eligible.

SAMPLE EXECUTION

- Username: student | Password: 1234
- Registration No: 101
- Name: Priya | Branch: CSE
- Marks: 88 | Total Classes: 100 | Attended: 85
- Output: Grade B | Attendance 85% | Eligible

ERROR HANDLING

- Provides Student Not Found, No Students Available and Invalid Choice messages.
- Input-format errors are not fully handled.
- Text entered where an integer is required can cause an exception.
- Future versions should use try-catch and reusable validation methods.

ADVANTAGES

- Simple and easy to operate.
- Low-cost and requires no external database.
- Uses core Java features.
- Demonstrates Java and DSA concepts in one application.
- Suitable for academic/training presentation.

LIMITATIONS

- Data exists only in runtime memory.
- Hard-coded credentials provide basic authentication.
- Console-only interface.
- Limited input validation.
- No database, GUI, reporting or multi-user role system.

FUTURE ENHANCEMENTS

- MySQL + JDBC for persistent storage.
- GUI or web application.
- Sorting by marks and branch filtering.
- Ranking and attendance reports.
- File export, multiple users, admin/student roles, dashboards and advanced validation.

SECURITY CONSIDERATIONS

- Hard-coded credentials are suitable only for demonstration.
- Deployed systems should use secure authentication and password hashing.
- Authorization and protected persistent storage should be added.
- Student academic information should be accessible only to authorized users.

LEARNING OUTCOMES

- Practical experience with classes, objects and constructors.
- Methods, static members, Scanner and ArrayList.
- Loops, conditions and switch-case.
- Modular design, testing, debugging and documentation.
- Understanding how a data structure supports an application workflow.

CONCLUSION

- Student Management System demonstrates a practical student-record workflow using Java and ArrayList.
- Supports authentication, CRUD-style operations, grade and percentage calculation, attendance evaluation and menu navigation.
- Provides a foundation for database, GUI, security, reporting, sorting and analytics.

VIVA / KEY QUESTIONS

- What is ArrayList and why is it used?
- Why is search $O(n)$ in this project?
- What OOP concepts are demonstrated?
- What is the attendance formula?
- Why is ArrayList add() amortized $O(1)$?
- How can the system be improved using MySQL and JDBC?

THANK YOU

- STUDENT MANAGEMENT SYSTEM
 - Java + Data Structures
 - Questions & Answers